

Customised Virtual File System

Platform required: Windows NT platform OR Linux Distributions.

Architectural requirement: Intel 32 bit processor.

User interface: Command User Interface

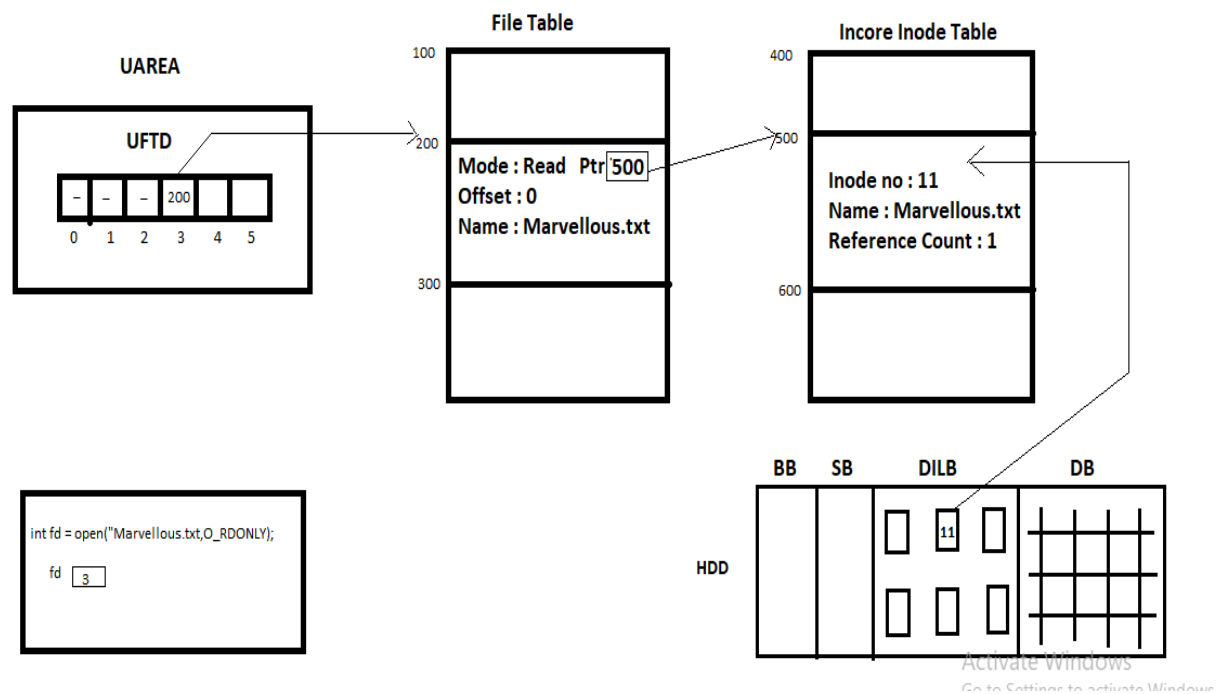
Technology Used : System Programming using C

Hardware requirements: Processor: Basic Processor

About Virtual File System

- In this project we emulate all data structures which are used by operating system to manage File system oriented tasks.
- As the name suggest its virtual because we maintain all records in Primary storage.
- In this project we create all data structures which required for File Subsystems as Inode, Inode Table, File Table, UAREA, User File Descriptor Table, Super block, Disk Inode List Block, Data Block, Boot Block etc.
- We provide all implementations of necessary system calls and commands of File subsystem as Open, Close, Read, Write, Lseek, Create, RM, LS, Stat, Fstat etc.
- While providing the implementations of all above functionality we use our own data structures by referring to Algorithms of UNNIX operating systems.
- By using this project we can get overview of UFS (UNIX File System) on any platform

Diagram of data structures used in the program:



1.What is meant by File System

A File System is a method used by the operating system to organize, store, retrieve, and manage files and directories on a storage device. It provides a logical structure for data storage and keeps track of file names, locations, permissions, and metadata.

2.Which file systems are used by Linux and Windows operating systems?

Linux : ext2, ext3, ext4 (Most commonly used), XFS, Btrfs

Windows : FAT32, exFAT, NTFS (Most commonly used), ReFS

3.What are the parts of the File System?

The CVFS consists of:

- Boot Block
- Super Block
- DILB (Disk Inode List Block)
- Inode
- File Table

- UAREA
- UFDT (User File Descriptor Table)
- Data Buffer (stores file data)

4. Explain UAREA and its contents.

UAREA stores information related to the running process.

Contents:

- Process Name
- UFDT (User File Descriptor Table)

Use:

It keeps track of all opened files of the current process..

5. Explain the use of the File Table and its contents.

The File Table stores information about an opened file.

Contents:

- Read Offset
- Write Offset
- Mode
- Pointer to Inode

Use:

It manages read/write operations and maintains the current position in the file.

6. Explain the use of In-Core Inode Table and its use.

An Inode stores the metadata of a file.

It contains:

- File Name
- Inode Number
- File Size
- Actual File Size
- File Type
- Permission
- Reference Count
- Pointer to Data Buffer

7. What is mean by inode?

An inode (Index Node) is a data structure that stores the information of a file except the actual file data. In this project, every file has its own inode

8. What are the contents of Superblock?

According to my project, the Super Block contains:

- Total Inodes
- Free Inodes

It keeps track of inode availability

9. What are the types of files?

According to my CVFS:

- Regular File
- Special File

10. What are the contents of the inode?

The inode contains:

- File Name
- Inode Number
- File Size
- Actual File Size
- File Type
- Reference Count
- Permission
- Buffer Pointer
- Next Pointer

11. What is the use of a Directory File?

A Directory File maps file names to inode numbers.

When a user searches for a file, the operating system first searches the directory, obtains the inode number, and then accesses the inode to retrieve the file metadata.

12. How operating system maintain security for files?

File security is maintained using permissions:

- Read (1)
- Write (2)
- Read + Write (3)

Before reading or writing, the program checks the file permission.

13. What happens when a user wants to open the file?

When the open command is executed:

- Check whether the mode is valid (1–3).
- Check whether the file exists using `IsFileExist()`
- Search the inode of the file.
- Check file permissions.
- Search for a free UFDT entry.
- Allocate memory for a new File Table.
- Initialize `ReadOffset`, `WriteOffset` and `Mode`
- Connect the File Table to the file's inode.
- Increment the inode's `ReferenceCount()`.
- Return the File Descriptor (FD)

14. What happens when a user calls `lseek()` system call?

`lseek()` changes the current `ReadOffset` of an opened file without reading or writing data. In my CVFS, it supports positioning from `START`, `CURRENT`, and `END`, after validating the file descriptor and offset.

15. What is the difference between a Library Function and a System Call?

A Library Function is a function provided by the C standard library that runs in user space. It performs common programming tasks and may internally call system calls when operating system services are needed. It is generally faster because it does not always switch to kernel mode. Examples: `printf()`, `scanf()`, `malloc()`.

A System Call is a function provided by the operating system that runs in kernel space. It allows a program to access system resources such as files, memory, and devices. It is slightly slower because it involves switching from user mode to kernel mode. Examples: `open()`, `read()`, `write()`, `close()`.

16. What is the use of this project?

The objective of the Customized Virtual File System (CVFS) project is:

- To understand the internal working of a file system.
- To implement a virtual file system completely in RAM (memory).
- To simulate file operations such as creat(), open(), close(), read(), write(), lseek(), stat(), unlink(), and ls.
- To gain practical knowledge of Boot Block, Super Block, Inode, File Table, UAREA, and UFDT.
- To understand how files are created, managed, accessed, and deleted without using the actual disk.

17. Is there any improvement needed in this project?

Yes. Currently, the project stores all data in RAM, so the data is lost when the program terminates. In the future, it can be improved by adding persistent storage, hierarchical directories, better security, and multi-user support.

18. What are the types of File Systems?

The main types of file systems are Disk File System, Network File System, Distributed File System, Virtual File System (VFS), and Memory-Based File System

19. What is the use of File table?

The File Table stores information about an opened file, such as ReadOffset, WriteOffset, Mode, and a pointer to the inode. It is used to manage file operations and keep track of the current position and access mode of the file.

20. Which things you refer to develop this project?

I referred to Linux File System concepts, Operating System concepts, C programming, POSIX system calls, and guidance from my mentor while developing this project.

21. On which platform we can execute this project?

This project can be executed on:

- Linux (Recommended, as it provides native support for POSIX libraries and system calls.)
- Windows, using a C compiler such as GCC/MinGW or any IDE that supports C programming (for example, VS Code with GCC).

Internal working of below system calls

1. open()

Checks whether the file exists and permissions are valid, creates a File Table entry, connects it to the inode, increments the Reference Count, and returns a File Descriptor.

2. close()

Decreases the Reference Count, resets the read/write offsets, frees the File Table, removes the UFDT entry, and closes the file

3. read()

Checks the File Descriptor and read permission, copies data from the file buffer to the user buffer, updates the Read Offset, and returns the number of bytes read.

4. write()

Checks the File Descriptor, write permission, and available space, writes data into the file buffer, updates the Write Offset and Actual File Size, and returns the number of bytes written.

5. lseek()

Changes the Read Offset of the opened file from START, CURRENT, or END after validating the File Descriptor and offset.

6. stat()

Searches the file by name and displays its metadata, such as File Name, Inode Number, File Size, Permission, Reference Count, and File Type.

7. chmod()

chmod() changes the permission bits stored in the file's inode after checking the required privileges.

8. unlink()

Deletes the file by freeing the file buffer and File Table, clearing the inode information, removing the UFDT entry, and increasing the number of free inodes.

Explain use of below commands

1. ls

Use: Displays the names of files and directories in the current directory.

2. ls - l

Use: Displays a detailed listing of files, including permissions, owner, size, and modification date

3. ls - a

Use: Displays all files, including hidden files (files starting with .)

4. rm

Use: Removes (deletes) files from the file system.

5. cat

Use: Displays the contents of a file. It can also be used to create or combine files.

6. cd

Use: Changes the current working directory.

7. chmod

Use: Changes the permissions of a file or directory.

8. cp

Use: Copies files or directories from one location to another.

9. df

Use: Displays disk space usage of the file system.

10. find

Use: Searches for files and directories based on name, type, or other criteria.

11. grep

Use: Searches for a specific word or pattern in a file.

12. ln

Use: Creates a hard link or symbolic link to a file.

13. mkdir

Use: Creates a new directory.

14. pwd

Use: Displays the absolute path of the current working directory.

15. touch

Use: Creates a new empty file or updates the timestamp of an existing file.

16. uname

Use: Displays information about the operating system and system kernel.

17. stat

Use: Displays detailed information about a file, such as inode number, size, permissions, owner, and timestamps. .

18. man

Use: Displays the manual (help documentation) for Linux commands and system calls. file.

19. mkfs

Use: Creates (formats) a file system on a storage device or partition.